Case Study 1

AKSTA Statistical Computing

Elias Anton Rozanics, Stefan Sick, Filip Cernak (Group 90)

The .Rmd and .html (alternatively the .pdf) should be uploaded in TUWEL by the deadline. Refrain from using explanatory comments in the R code chunks but write them as text instead. Please consider the AI Guidelines as specified in the organizational slides (you can use the table). Points will be deducted if the submitted file is not in a decent form; answers should be answering the concrete questions and not discuss topics in general - please be mindful of our tutors' time and the size of the course.

Table 1: You can use and extend this table to specify your AI usage (if used at all).

AI usage	**Task 1**	**Task 2**	**Task 3**	**Task 4**	**Task 5**	**Task 6**
Correcting faulty code	no	no	no	no	no	little
Improving existing code	no	no	no	no	no	no
Generating new code	no	no	no	no	no	no
Layout and formatting	no	no	no	no	no	no
Text correction	little	no	no	no	no	no
Help with Latex expressions	no	little	little	little	no	no
Explanation of R base function	no	no	little	little	little	no

1. Ratio of Fibonacci numbers (1 point)

a.

Write two different R functions which return the sequence $r_i = F_{i+1}/F_i$ for $i = 1, \ldots, n$ where F_i is the ith Fibonacci number, once using `for` and once using `while`.

```
#for loop

fibonacci_for <- function(n) {
  F <- numeric(n + 1)
  F[1] <- 1
  F[2] <- 1

  for(i in 3:(n+1)){
    F[i] <- F[i-1] + F[i-2]
  }

  r <- numeric(n)
  for (i in 1:n) {
    r[i] <- F[i + 1] / F[i]
  }

  return(r)
```

```r
}

#while loop

fibonacci_while <- function(n) {
  F <- numeric(n + 1)
  F[1] <- 1
  F[2] <- 1

  i <- 3
  while (i <= n + 1) {
    F[i] <- F[i - 1] + F[i - 2]
    i <- i + 1
  }

  r <- numeric(n)
  i <- 1
  while (i <= n) {
    r[i] <- F[i + 1] / F[i]
    i <- i + 1
  }

  return(r)
}

fibonacci_for(10)
```

```
##  [1] 1.000000 2.000000 1.500000 1.666667 1.600000 1.625000 1.615385 1.619048
##  [9] 1.617647 1.618182
```

```r
fibonacci_while(10)
```

```
##  [1] 1.000000 2.000000 1.500000 1.666667 1.600000 1.625000 1.615385 1.619048
##  [9] 1.617647 1.618182
```

We can see that both of the functions give us the same results.

b.

Benchmark the two functions for $n = 200$ and $n = 2000$ (you can use package **microbenchmark** or package **bench** for this purpose). Which function is faster?

```r
library(microbenchmark)

#benchmark for 200

microbenchmark(
  for_function_200 = fibonacci_for(200),
  while_function_200 = fibonacci_while(200)
)
```

```
## Unit: microseconds
##                expr  min   lq   mean median    uq  max neval
##    for_function_200 33.6 34.6 36.893  35.00 35.80 89.1   100
##  while_function_200 49.1 50.0 51.547  50.65 51.65 85.4   100
```

```r
#benchmark for 2000

microbenchmark(
  for_function_2000 = fibonacci_for(2000),
  while_function_2000 = fibonacci_while(2000)
)
```

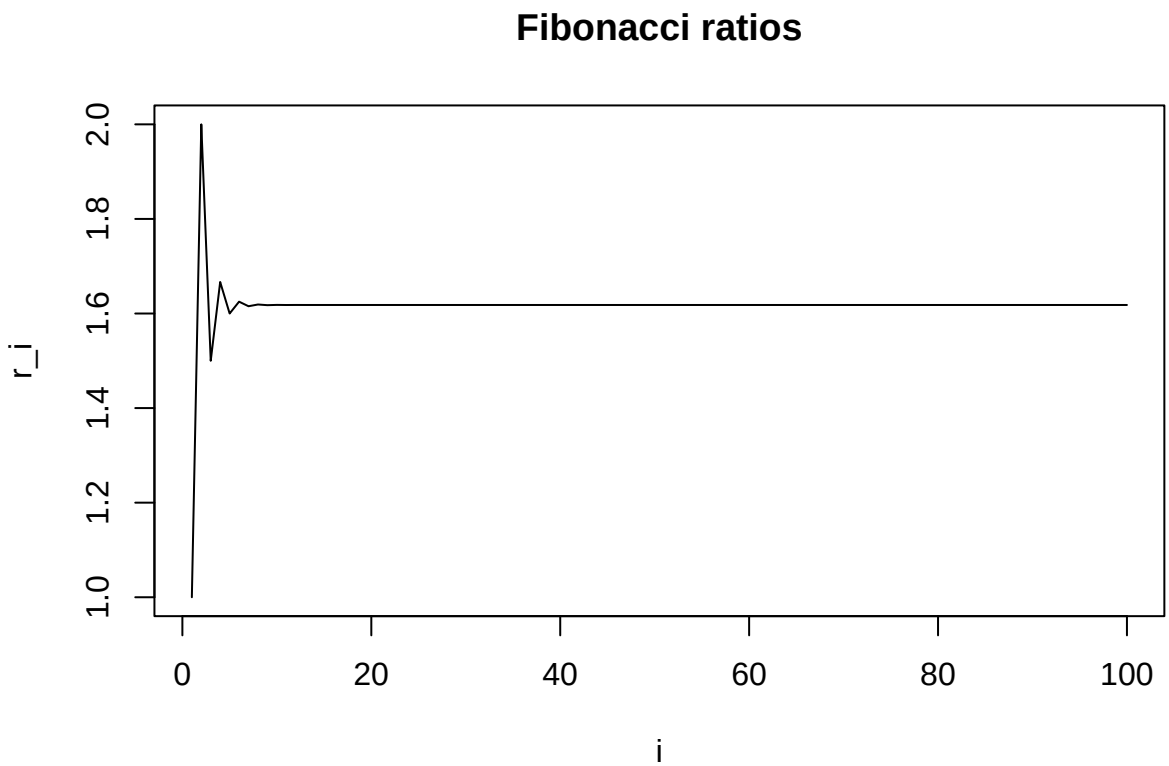
```
## Unit: microseconds
##                 expr   min     lq    mean median    uq    max neval
##    for_function_2000 317.6 333.15 375.661 337.70 345.9 2545.5   100
##  while_function_2000 482.6 490.90 561.574 496.95 509.7 3360.0   100
```

In this case, the microbenchmark() function runs by default each expression 100 times. We can observe that in both cases (n=200 and n=2000), the function using for is faster. The median time for the function using for is around 1.5 times faster than the function using while for both n=200 and n=2000. We mainly look at the median here instead of the mean as some of the max values are way too large compared to the median, and so this could have an effect on the mean.

c.

Plot the sequence for $n = 100$. For which n does it start to stabilize to a value? What number does the sequence converge to?

```r
ratio_100 <- fibonacci_for(100)

plot(ratio_100, type = "l",
     xlab = "i",
     ylab = "r_i",
     main = "Fibonacci ratios")
```



When looking at the plot, we can see that the sequence starts to stabilize at around n=7 or n=8. For larger n, the plot looks completely flat, which shows that the sequence converges. The sequence converges towards a value around 1.62 ($\approx$ 1.61803 to be exact). This number is also called the Golden Ratio, which is a well known limit of the ratio of consecutive Fibonacci numbers.

2. Gamma function (1 point)

a.

Write a function to compute the following for n a positive integer using the `gamma` function in base R.

$$\rho_n = \frac{\Gamma((n-1)/2)}{\Gamma(1/2)\Gamma((n-2)/2)}$$

```
rho_n <- function(n){
  gamma((n - 1)/2) / (gamma(1/2) * gamma((n - 2)/2))
}
```

b.

Try $n = 2000$. What do you observe? Why do you think the observed behavior happens?

```
rho_n(2000)
```

```
## [1] NaN
```

When evaluating our function for n=2000, we get NaN (not a number). This means that we have an undefined operation that is going on. Since the arguments $\frac{n-1}{2}$ and $\frac{n-2}{2}$ are very large for n=2000, and the Gamma

function grows even faster than exponentially, we have in this case Inf/Inf, which gives us NaN. This happens because the values become so large before the division in both the numerator and denominator that R cannot represent them and so stores the values as Inf.

c.

Write an implementation which can also deal with large values of $n > 1000$.

To deal with large values of n > 1000, we will work with the logarithm of gamma using the function lgamma() and then exponentiate the final result.

```
rho_large_n <- function(n) {
  exp(lgamma((n - 1)/2) - lgamma(1/2) - lgamma((n - 2)/2))
}

rho_large_n(2000)
```

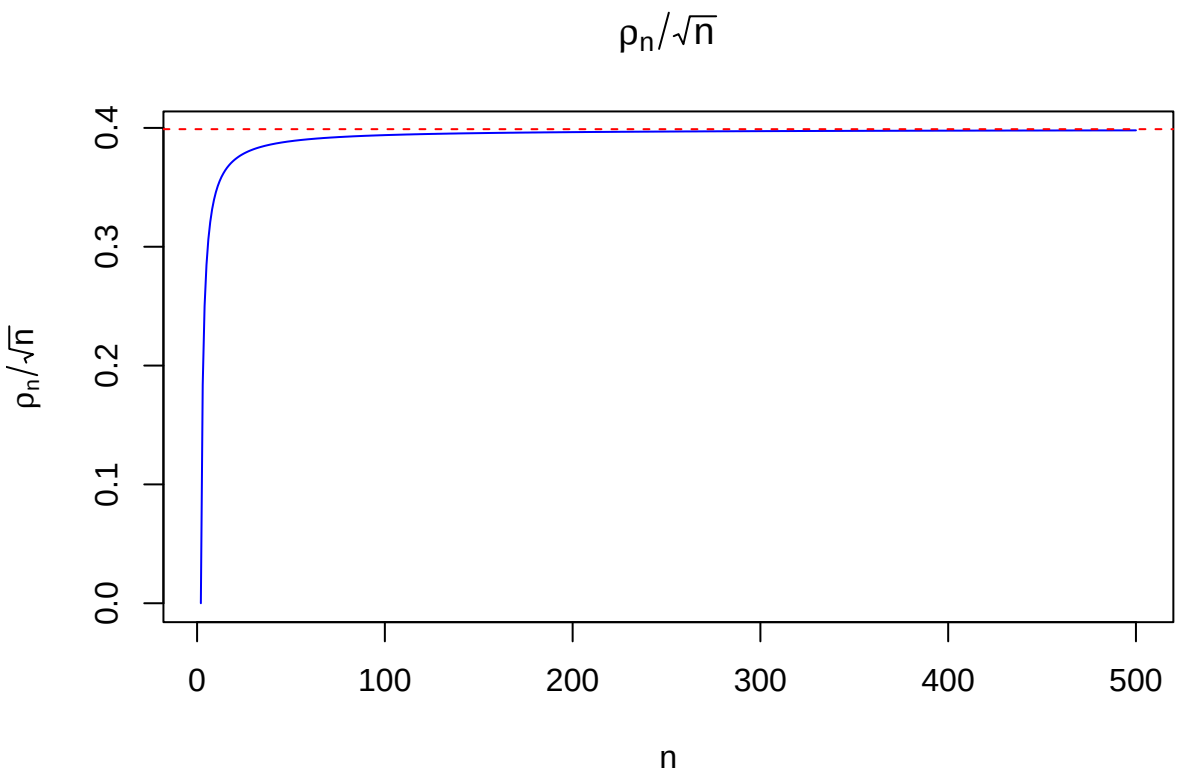
```
## [1] 17.83009
```

We can now see that the stabilized function with the logarithm and exponentiation gives us a finite result even for very large n. And so instead of working with the very large Gamma values and then dividing (which leads to Inf/Inf), we work in the log space. Because log(Gamma) grows much slower than Gamma, the values we obtain are manageable for R to work with. We then subtract these log-gamma values (instead of dividing in the original formula) and then exponentiate, which gives us a finite result.

d.

Plot $\rho_n/\sqrt{n}$ for different values of n. Can you guess the limit of $\rho_n/\sqrt{n}$ for $n \to \infty$?

```
n_values <- 2:500
rho_values <- sapply(n_values, rho_large_n)
plot(n_values, rho_values / sqrt(n_values), type = "l", col = "blue",
     main = expression(rho[n]/sqrt(n)),
     xlab = "n", ylab = expression(rho[n]/sqrt(n)))
abline(h = 1/sqrt(2*pi), col = "red", lty = 2)
```



To estimate the limit of

$$\frac{\rho_n}{\sqrt{n}} \quad \text{for } n \to \infty,$$

let's have a closer look at the gamma functions from ρ_n. We know that $\Gamma\left(\frac{1}{2}\right) = \sqrt{\pi}$. For the remainder of the Gamma terms, we will use Stirling's approximation formula:

$$\Gamma(x) \approx \sqrt{2\pi}\, x^{x-\frac{1}{2}} e^{-x}$$

When we apply this to our 2 remaining Gamma terms in ρ_n, we get:

$$\Gamma\left(\frac{n-1}{2}\right) \approx \sqrt{2\pi}\left(\frac{n-1}{2}\right)^{\frac{n-1}{2}-\frac{1}{2}} e^{-\frac{n-1}{2}} = \sqrt{\frac{4\pi}{n-1}}\left(\frac{n-1}{2e}\right)^{\frac{n-1}{2}}$$

and

$$\Gamma\left(\frac{n-2}{2}\right) \approx \sqrt{2\pi}\left(\frac{n-2}{2}\right)^{\frac{n-2}{2}-\frac{1}{2}} e^{-\frac{n-2}{2}} = \sqrt{\frac{4\pi}{n-2}}\left(\frac{n-2}{2e}\right)^{\frac{n-2}{2}}$$

After we plug these back into ρ_n, we then simplify the square root terms and also the exponential terms. After these simplifications, we get:

$$\rho_n \approx \sqrt{\frac{n-2}{2\pi}}$$

Finally we divide this expression by $\sqrt{n}$ and by taking the limit for $n \to \infty$, we obtain:

$$\frac{\rho_n}{\sqrt{n}} \approx \sqrt{\frac{n-2}{2\pi n}} = \sqrt{\frac{1}{2\pi}\cdot\frac{n-2}{n}} \overset{n\to\infty}{\longrightarrow} \sqrt{\frac{1}{2\pi}} = \frac{1}{\sqrt{2\pi}}.$$

The plot confirms this, as we can observe convergence of $\frac{\rho_n}{\sqrt{n}}$ towards the red line, which represents $\frac{1}{\sqrt{2\pi}}$.

3. Weak law of large numbers (1.5 points)

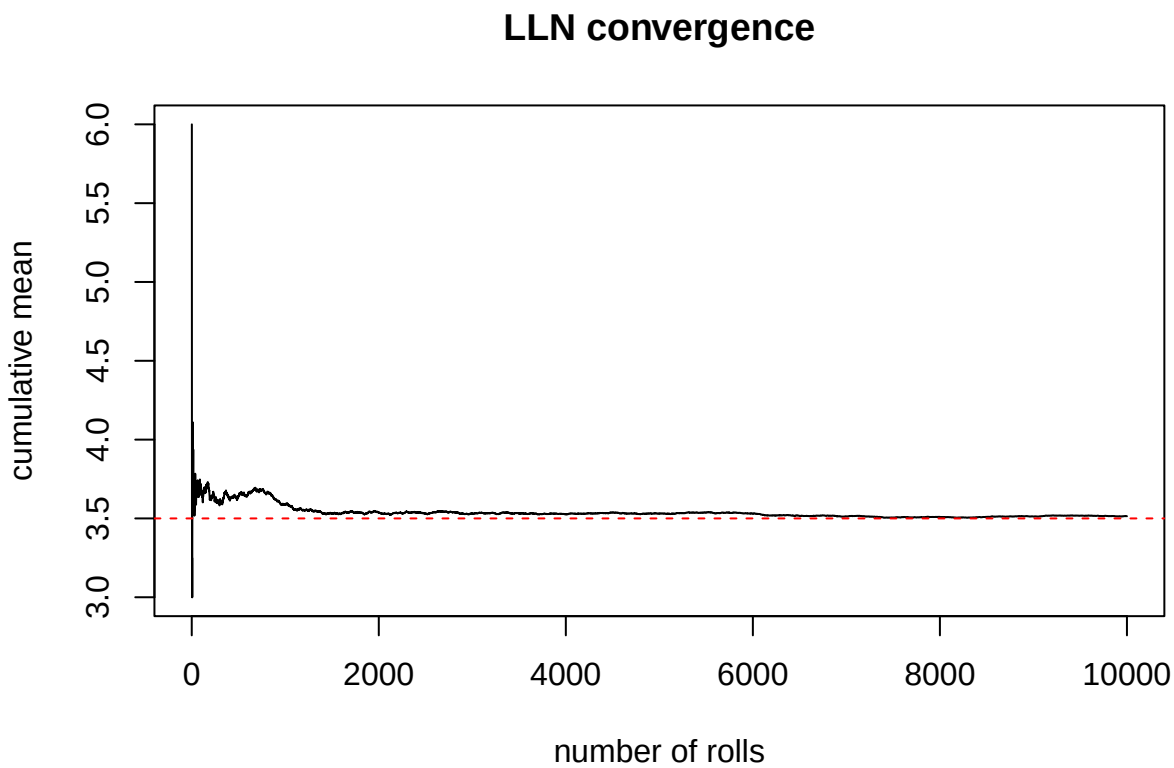
The law of large numbers states that if you repeat an experiment independently a large number of times and average the result, what you obtain should be close to the expected value. Formally, the weak law of large numbers states that if $X_1, X_2, \ldots X_n$ are independently and identically distributed (iid) random variables with a finite expected value $E(X_i) = \mu < \infty$, then, for any $\epsilon > 0$ $\lim_{n\to\infty} P(|\bar{X}_n - \mu| > \epsilon) = 0$ where $\bar{X}_n = \frac{X_1+X_2+\ldots+X_n}{n}$ is the sample mean (the sample mean converges in probability to the expected value).

a.

- In R, simulate rolling a fair 6-sided die $n = 10\,000$ times using vectorized functions.
- After each roll, compute the cumulative mean of all previous rolls.
- Plot the running average against the number of rolls and comment on the result.

```r
roll_die <- function(num_rolls=10000, num_sides=6) {
  rolls <- sample(1:num_sides, size=num_rolls, replace=TRUE)
  return (cumsum(rolls) / 1:num_rolls)
}

plot_rolls <- function(cum_means) {
  if (!is.vector(cum_means)) {
    cum_means <- t(cum_means)
  }
  matplot(cum_means, type="l",
     main="LLN convergence",
     xlab="number of rolls",
     ylab="cumulative mean")
}
```

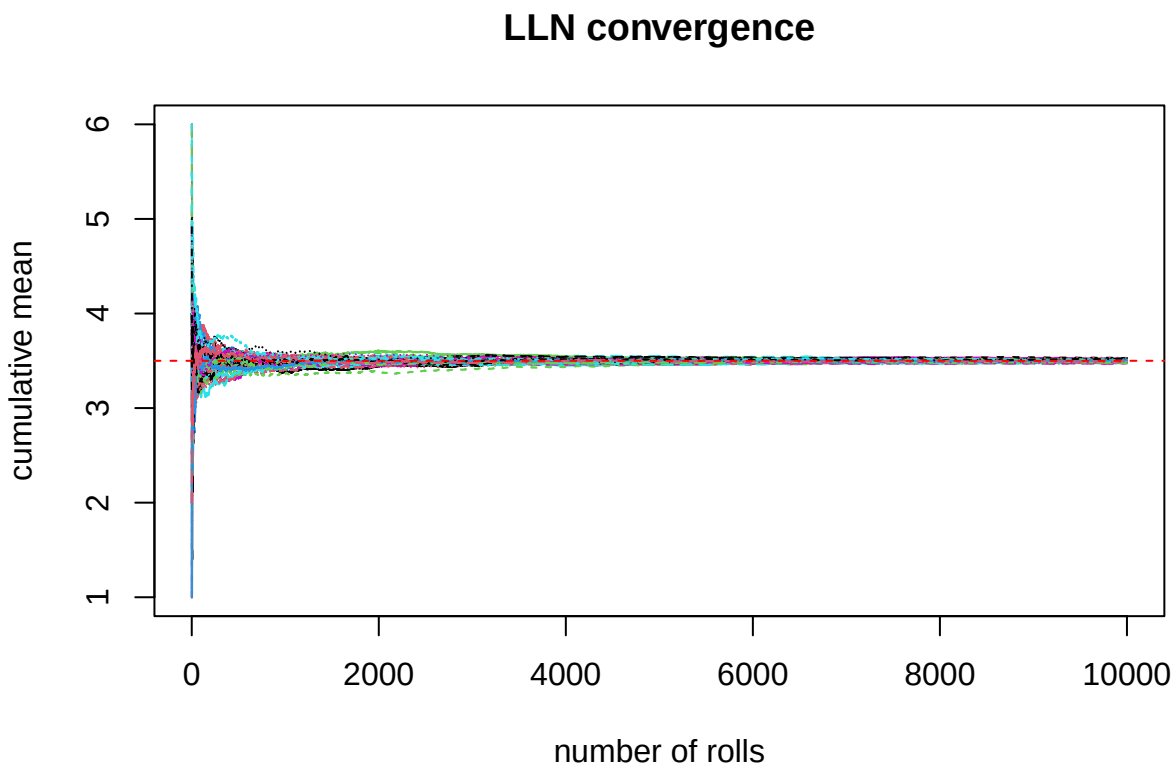
```r
n <- 10000
k <- 6
cum_mean <- roll_die(n, k)

plot_rolls(cum_mean)
abline(h=3.5, col = "red", lty=2) # lty 2 -> dashed
```



b.

- Replicate the die-rolling experiment $M = 50$ times using a for loop. This will result in 50 paths which you can plot against the number of rolls.
- Plot all 50 paths on the same graph to observe the variation in the running averages.

```
M <- 50
histories <- matrix(NA, nrow = M, ncol = n)
for (i in 1:M) {
  histories[i,] <- roll_die(n, k)
}
plot_rolls(histories)
abline(h=3.5, col = "red", lty=2) # lty 2 -> dashed
```



c.

What is the theoretical expected value of a fair 6-sided die?

$$E(X) = \sum_{k=1}^{n} k \cdot P(k)$$

$$\forall k : P(k) = \frac{1}{n}$$

therefore:

$$E(X) = \frac{1}{n} \sum_{k=1}^{n} k$$

In this case, n=6

```
mean(1:6)
```

```
## [1] 3.5
```

d.

Comment on how the running mean of the paths behaves relative to the expected value as n increases.

As n increases, the running means of all paths have less variance and converge to the expected value.

e.

- Modify the code in b. to use a while loop.
- Stop the loop when the proportion of replications where the difference between the running average and the expected value lies outside the interval $[-\epsilon, \epsilon]$ is less than 0.05. Use $\epsilon = 0.01$.
- How high is n at the end of the loop?

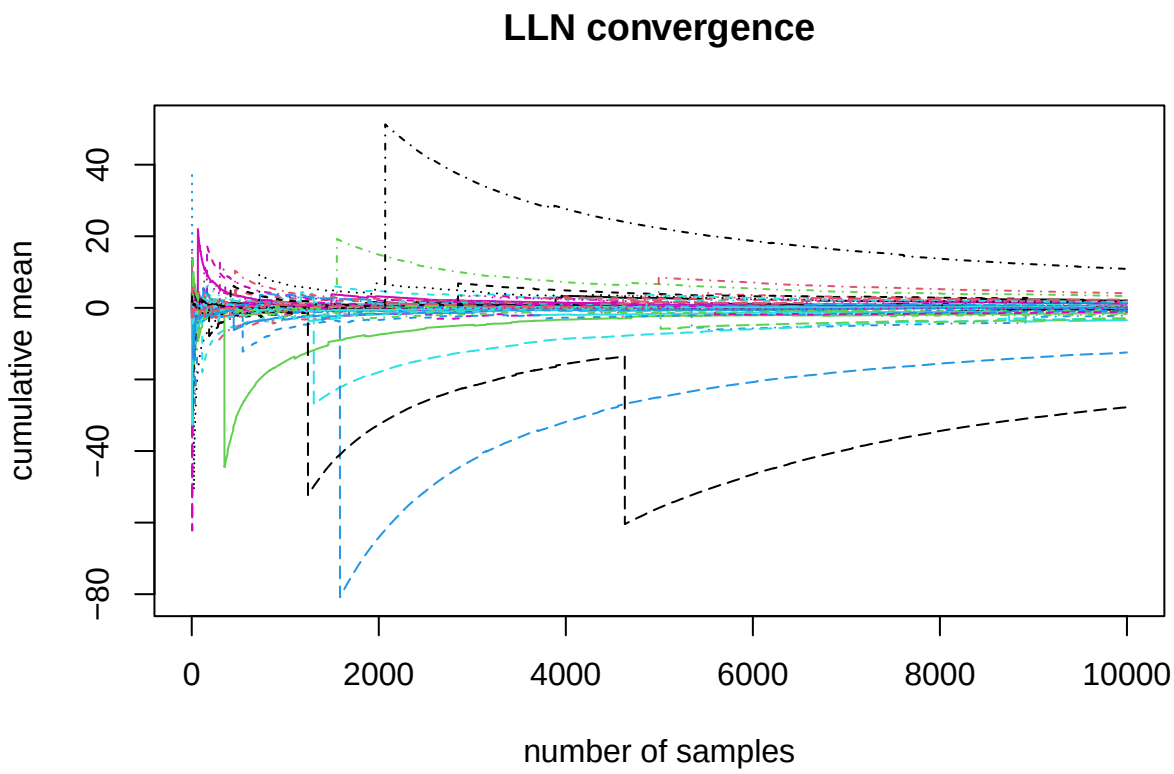
```
roll_until_convergence <- function(M = 50, epsilon = 0.01, target = 0.05, expected_value=3.5) {
  n <- 0
  means <- rep(0, M)
  prop_outside <- 1

  while (prop_outside >= target) {
    n <- n + 1

    rolls <- sample(1:6, M, replace = TRUE)
    means <- ((n-1)*means + rolls) / n

    # mean of booleans (0,1) gives proportion
    prop_outside <- mean(abs(means - expected_value) > epsilon)
  }

  return(n)
}
n_convergence <- roll_until_convergence()
print(n_convergence)
```

```
## [1] 106460
```

f.

- Repeat the analysis in b, but instead of simulating from a discrete uniform distribution, simulate from a continuous standard cauchy distribution using `rcauchy()`. The (standard) cauchy distribution arises from the ratio of two independent mean zero (standard) normally distributed random variables.
- Plot the running mean against increasing n. Comment on whether the weak Law of Large Numbers seems to hold for this distribution. Explain why or why not.

```
histories_cauchy <- matrix(NA, nrow = M, ncol = n)
for (i in 1:M) {
  cauchy_samples <- rcauchy(n, location=0, scale=1)
  histories_cauchy[i,] <- (cumsum(cauchy_samples) / 1:n)
}
matplot(t(histories_cauchy), type="l",
        main="LLN convergence",
        xlab="number of samples",
        ylab="cumulative mean")
```



The LLN does not hold in this case because the Cauchy distribution does not have a finite mean.

4. Central limit theorem (1.5 points)

The central limit theorem states that, under certain conditions, the sum of a large number of random variables is approximately normal. Consider the case where $X_1, X_2, \ldots, X_n$ are i.i.d. random variables with finite expected values ($E(X_i) = \mu < \infty$) and variances ($Var(X_i) = \sigma^2 < \infty$). The central limit theorem states that the cumulative distribution function (CDF) of the normalized random variable

$$Z_n = \frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}}$$

converges to the standard normal CDF as $n \to \infty$.

a.

Derive the expected value of the sample mean $\bar{X}_n = \frac{X_1+X_2+\ldots+X_n}{n}$.

sample mean

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i,$$

the expected value is:

$$E(\bar{X}_n) = E\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n}\sum_{i=1}^{n} E(X_i) = \frac{1}{n}\sum_{i=1}^{n} \mu = \mu$$

b.

Derive the variance of the sample mean $\bar{X}_n = \frac{X_1+X_2+\ldots+X_n}{n}$.

variance of the sample mean (constant factor gets squared):

$$\mathrm{Var}(\bar{X}_n) = \mathrm{Var}\left(\frac{1}{n}\sum_{i=1}^{n} X_i\right) = \frac{1}{n^2}\,\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right)$$

Since the X_i are independent and identically distributed:

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i) = n\,\mathrm{Var}(X)$$

because $\mathrm{Var}(X) = \sigma^2$:

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = n\sigma^2$$

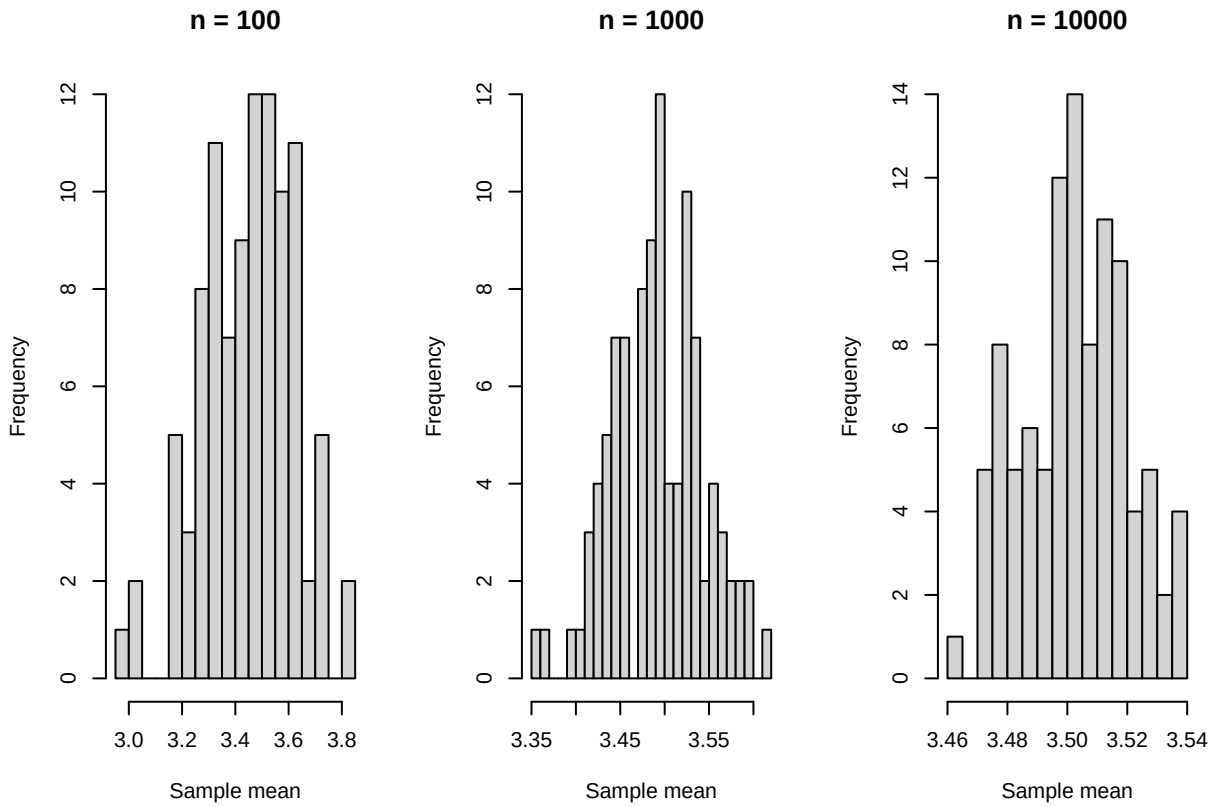
$$=> \mathrm{Var}(\bar{X}_n) = \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}$$

c.

The central limit theorem makes few assumption on the distribution of the random variables X_i. Consider again the die-rolling experiment where X_i is the random variable giving the number in roll i.

- Simulate Die Rolls: For each value of n (where $n = \{100, 1000, 10000\}$), simulate rolling a die n times. Record the mean of the n rolls.
- Repeat the above process $M = 100$ times for each value of n. This will give you 100 sample means for each n.
- Plot a histogram of the 100 sample means for each value of n. Observe the shape of the histograms.

```r
ns <- c(100, 1000, 10000)
M <- 100

par(mfrow = c(1,3))

for (n in ns) {
  sample_means <- numeric(M)

  for (i in 1:M) {
    rolls <- sample(1:6, n, replace = TRUE)
    sample_means[i] <- mean(rolls)
  }

  hist(sample_means,
       main=paste("n =", n),
       xlab = "Sample mean",
       breaks = M/5
  )
}
```



- Discuss how the distribution of the sample means changes as n increases. Comment on whether the histograms appear to approach a normal distribution as n, illustrating the Central Limit Theorem.

The sample mean is a sum of iid. random variables with mean and variance $< \infty$, so the distribution approaches a normal distribution. However, this isn't very visible here because the distribution of a fair die is already symmetric. Therefore, even for low n, the sample means appear approximately normal. What is also visible from the x-axis is the fact that the variance gets smaller with increasing n.

d.

Compute expected value μ and the variance σ^2 for the random variable X_i (giving the number in die-roll i), which follows a discrete uniform distribution.

The expected value of a fair 6-sided die is 3.5 -> (see 3c)

The variance is:

$$\text{Var}(X_i) = E(X_i^2) - (E(X_i))^2$$

With $\frac{1}{6}\sum_{k=1}^{6} k^2 = \frac{91}{6}$ we can insert the values and get:

$$\text{Var}(X_i) = \frac{91}{6} - 3.5^2 = \frac{35}{12}$$

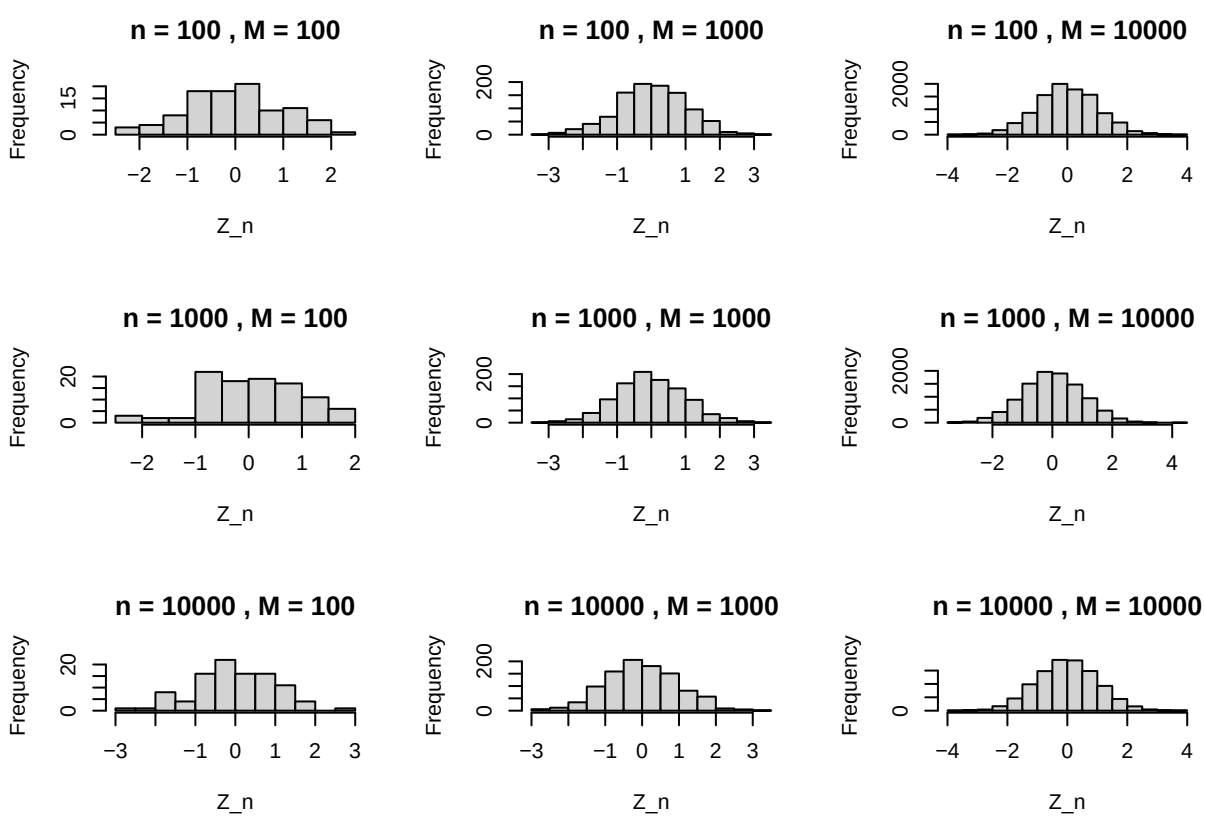
e.

Now that you computed μ and σ^2, consider Z_n.

- For each value of n (where $n = \{100, 1000, 10000\}$) and for $M = \{100, 1000, 10000\}$ replications, simulate the rolling of n dice and calculate the Z_n.

- Plot histograms of Z_n for each combination of n and M.

```r
ns <- c(100, 1000, 10000)
Ms <- c(100, 1000, 10000)

expected_value <- 3.5
std <- sqrt(35/12)

par(mfrow = c(length(ns), length(Ms)))

for (n in ns) {
  for (M in Ms) {

    Z <- numeric(M)

    for (i in 1:M) {
      rolls <- sample(1:6, n, replace = TRUE)
      sample_mean <- mean(rolls)
      Z[i] <- (sample_mean - expected_value) / (std / sqrt(n))
    }

    hist(Z,
         main = paste("n =", n, ", M =", M),
         xlab = "Z_n")
  }
}
```



- Discuss how the distribution of Z_n changes as n and M increase. Comment on whether the distributions of Z_n appear to approach a normal distribution, illustrating the Central Limit Theorem.

As M increases, the histograms become smoother and less noisy because more values are used to approximate the distribution of Z_n. As n increases, the distribution of Z_n approaches the standard normal distribution according to the CLT. However, this is not very visible in these plots because the underlying distribution is symmetric and can only take on values in 1:6, so even for small n the approximation is already close to a standard normal distribution, which is visible if M is of sufficient size.

5. Readable and efficient code (1 points)

Read over the code below.

a.

Explain (in text) what the code does.

Chunk 1: The code initializes two normally distributed arrays of length 100, which are used to perform linear regression. After each prediction the residuals are used to be the next dependent variable and changed to a sinus wave with the amplitude increased and the mean slightly shifted. The if tests make sure there is enough noise in the data.

Chunk 2: The code initializes one normally distributed array of length 1000, and one where 2 plus some noise from another normal distribution are added. The rows are manually split into four blocks of length 250. Similar to a cross validation each of the blocks is used for prediction while a model is trained on the remaining 75% of the data. For each run the RMSE is calculated and then stored in a vector.

b.

Explain (in text) what you would change to make the code more readable.

To get rid of the repetitive parts the a function should be created that can be called in a loop afterwards.

c.

Change the code according to b. and wrap it in a function. This function should have at most 10 lines (without adding commands to more lines such as `x <- 1; y <- 2`. Such commands will count as 2 lines!). Check that the function called on the same input outputs the same as the provided code.

```r
set.seed(1)
x <- rnorm(100)
z <- rnorm(100)
if (sum(x >= .001) < 1) {
  stop("step 1 requires 1 observation(s) with value >= .001")
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- sin(r) + .01
if (sum(x >= .002) < 2) {
  stop("step 2 requires 2 observation(s) with value >= .002")
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- 2 * sin(r) + .02
if (sum(x >= .003) < 3) {
  stop("step 3 requires 3 observation(s) with value >= .003")
```

```r
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- 3 * sin(r) + .03
if (sum(x >= .004) < 4) {
  stop("step 4 requires 4 observation(s) with value >= .004")
}
fit <- lm(x ~ z)
r <- fit$residuals
x <- 4 * sin(r) + .04
x
```

```r
change_residuals <- function(x, z, index){
    if (sum(x >= index * 0.001) < index) {
        stop(sprintf("Step %d failed: threshold %.3f not met.", index, index*0.001))}
    fit <- lm(x ~ z)
    r <- fit$residuals
    return (index * sin(r) + (index * 0.01))
    }

set.seed(1)
x <- rnorm(100)
z <- rnorm(100)

for (i in 1:4){
    x <- change_residuals(x, z, i)
}
x
```

```r
set.seed(1)
x <- rnorm(1000)
y <- 2 + x + rnorm(1000)
df <- data.frame(x, y)

cat("Step", 1, "\n")
fit1 <- lm(y ~ x, data = df[-(1:250),])
p1 <- predict(fit1, newdata = df[(1:250),])
r <- sqrt(mean((p1 - df[(1:250),"y"])^2))

cat("Step", 2, "\n")
fit2 <- lm(y ~ x, data = df[-(251:500),])
p2 <- predict(fit2, newdata = df[(251:500),])
r <- c(r, sqrt(mean((p2 - df[(251:500),"y"])^2)))

cat("Step", 3, "\n")
fit3 <- lm(y ~ x, data = df[-(501:750),])
p3 <- predict(fit3, newdata = df[(501:750),])
r <- c(r, sqrt(mean((p3 - df[(501:750),"y"])^2)))

cat("Step", 4, "\n")
fit4 <- lm(y ~ x, data = df[-(751:1000),])
p4 <- predict(fit4, newdata = df[(751:1000),])
r <- c(r, sqrt(mean((p4 - df[(751:1000),"y"])^2)))
r
```

```r
get_rmse <- function(df, num_folds, index){
    num_rows <- nrow(df) / num_folds
    start_row <- (index - 1) * num_rows + 1
    end_row <- index * num_rows

    cat(sprintf("Step %d\n", index))

    fit <- lm(y ~ x, data = df[-(start_row:end_row),])
    p <- predict(fit, newdata = df[(start_row:end_row),])
    r <- sqrt(mean((p - df[(start_row:end_row),"y"])^2))

    return (r)
}

set.seed(1)
x <- rnorm(1000)
y <- 2 + x + rnorm(1000)
df <- data.frame(x, y)

fold_num <- 4
result <- c()
for (i in 1:fold_num){
    result <- c(result, get_rmse(df, fold_num, i))
}
result
```

6. Measuring and improving performance (2 points)

Have a look at the code of the function below. It is a function for performing a Kruskal-Wallis test, a robust non-parametric method for testing whether samples come from the same distribution. (Note: we assume no missing values are present in `x`).

```r
kwtest <- function (x, g, ...)
{
    if (is.list(x)) {
        if (length(x) < 2L)
            stop("'x' must be a list with at least 2 elements")
        if (!missing(g))
            warning("'x' is a list, so ignoring argument 'g'")
        if (!all(sapply(x, is.numeric)))
            warning("some elements of 'x' are not numeric and will be coerced to numeric")
        k <- length(x)
        l <- lengths(x)
        if (any(l == 0L))
            stop("all groups must contain data")
        g <- factor(rep.int(seq_len(k), l))
        x <- unlist(x)
    }
    else {
        if (length(x) != length(g))
            stop("'x' and 'g' must have the same length")
        g <- factor(g)
        k <- nlevels(g)
```

```r
        if (k < 2L)
            stop("all observations are in the same group")
    }
    n <- length(x)
    if (n < 2L)
        stop("not enough observations")
    r <- rank(x)
    TIES <- table(x)
    STATISTIC <- sum(tapply(r, g, sum)^2/tapply(r, g, length))
    STATISTIC <- ((12 * STATISTIC/(n * (n + 1)) - 3 * (n + 1))/(1 -
        sum(TIES^3 - TIES)/(n^3 - n)))
    PARAMETER <- k - 1L
    PVAL <- pchisq(STATISTIC, PARAMETER, lower.tail = FALSE)
    names(STATISTIC) <- "Kruskal-Wallis chi-squared"
    names(PARAMETER) <- "df"
    RVAL <- list(statistic = STATISTIC, parameter = PARAMETER,
        p.value = PVAL, method = "Kruskal-Wallis rank sum test")
    return(RVAL)
}
```

a.

Write a pseudo code outlining what the function does.

If x is list: x must be >= 2 if there is also a g provided, warn that labels will be ignored check that all elements are numeric check that all groups are not empty create one factor list for all groups(labels) create a vector out of the list

Else: check that number of x == number of labels convert labels to factor check that number of levels >= 2

Check If observations >= 2

Convert all x values to their ranks Store the size of all ties in a list

Calculate the sum of all ranks in a group Apply Kruskal-Wallis

Calculate the significance with chi-squared

Return result

b.

For example data, call the function in two ways: once using `x` as a list and once using `x` as a vector with a corresponding `g` argument. Ensure that the two different function calls return the same thing by aligning the inputs.

```r
x_list <- list(grp1 = c(1, 6, 2, 5),
        grp2 = c(11, 16, 12, 15),
        grp3 = c(21, 26, 22, 25))
x_vec <- c(1, 6, 2, 5, 11, 16, 12, 15, 21, 26, 22, 25)
g_vec <- rep(c("grp1", "grp2", "grp3"), each = 4)

kw_list <- kwtest(x_list)
kw_list
```

```
## $statistic
## Kruskal-Wallis chi-squared
##                   9.846154
```

```
##
## $parameter
## df
##  2
##
## $p.value
## [1] 0.007276706
##
## $method
## [1] "Kruskal-Wallis rank sum test"
```

```r
kw_vec <- kwtest(x_vec, g_vec)
kw_vec
```

```
## $statistic
## Kruskal-Wallis chi-squared
##                   9.846154
##
## $parameter
## df
##  2
##
## $p.value
## [1] 0.007276706
##
## $method
## [1] "Kruskal-Wallis rank sum test"
```

c.

Make a faster version of `kwtest()` that only computes the Kruskal-Wallis **test statistic** when the input is a numeric variable `x` and a variable `g` which gives the group membership. You can try simplifying the function above or by coding from the mathematical definition (see https://en.wikipedia.org/wiki/Kruskal%E2%80%93Wallis_one-way_analysis_of_variance). This function should also perform some checks to ensure the correctness of the inputs (use `kwtest()` as inspiration).

```r
x_vec <- c(1, 6, 2, 5, 11, 16, 12, 15, 21, 26, 22, 25)
g_vec <- rep(c("grp1", "grp2", "grp3"), each = 4)

kw_fast <- function(x, g){
     if (length(x) != length(g))
            stop("'x' and 'g' must have the same length")
    g <- factor(g)
    k <- nlevels(g)
    if (k < 2L)
        stop("all observations are in the same group")

    n <- length(x)
    if (n < 2L)
        stop("not enough observations")

    r <-data.table::frank(x, ties.method = "average")
    g_sums <- collapse::fsum(r, g)
    g_sizes <- collapse::fcount(g)$N
```

```r
    H <- (12 / (n * (n + 1))) * sum(g_sums^2 / g_sizes) - 3 * (n + 1)

    t <- collapse::fcount(x)$N
    t <- t[t > 1]
    tie_correction <- 1 - (sum(t^3 - t) / (n^3 - n))

    return(H / tie_correction)
}


kw_fast_test <- kw_fast(x_vec, g_vec)
kw_fast_test
```

```
## [1] 9.846154
```

d.

Consider the following scenario. You have samples available from multiple experiments $m = 1000$ where you collect the numerical values for the quantity of interest `x` and the group membership for n individuals. The first 20 individuals in each sample belong to group 1, the following 20 individuals in each sample belong to group 2, the last 10 individuals in each sample belong to group 3. Use the following code to simulate such a data structure:

```r
set.seed(1234)
m <- 1000 # number of repetitions
n <- 50   # number of individuals
X <- matrix(rt(m * n, df = 10), nrow = m)
grp <- rep(1:3, c(20, 20, 10))
```

Write a function which performs the Kruskal-Wallis test using the function `stats:::kruskal.test.default()` for m repeated experiments and returns a vector of m test statistics. The input of this function are a matrix `X` with m rows and n columns and a vector `g` which gives the grouping.

```r
stats_kw <- apply(X, 1, function(row){stats:::kruskal.test.default(row, grp)$statistic})
stats_kw
```

```
##    [1] 3.846353e+00 5.463529e-01 3.335294e+00 1.878353e+00 1.508706e+00
##    [6] 2.677176e+00 7.181176e-01 5.186588e+00 2.265882e-01 2.431059e+00
##   [11] 2.058353e+00 5.671294e+00 2.172941e+00 2.488235e+00 3.172235e+00
##   [16] 2.443059e+00 6.416471e-01 1.243059e+00 1.274824e+00 1.821882e+00
##   [21] 2.308235e+00 1.478118e+00 7.771765e-01 4.748235e-01 1.718353e+00
##   [26] 4.371765e-01 1.183482e+01 3.204706e-01 2.191765e+00 4.784706e+00
##   [31] 2.124706e-01 1.711059e+00 2.386588e+00 3.329412e-01 4.623059e+00
##   [36] 9.428235e-01 5.238353e+00 4.259529e+00 8.004706e-01 1.054118e-01
##   [41] 6.818824e-01 2.971765e-01 2.913647e+00 1.607529e+00 1.923529e+00
##   [46] 1.294824e+00 3.664706e+00 8.195294e-01 6.252941e+00 1.003059e+00
##   [51] 5.761882e+00 1.265412e+00 1.729412e-01 1.725882e+00 2.921882e+00
##   [56] 2.894118e-02 7.630118e+00 2.033647e+00 5.465882e-01 5.717647e-02
##   [61] 6.543529e+00 1.167529e+00 6.416471e-01 2.325176e+00 8.658824e-02
##   [66] 4.298824e-01 1.719529e+00 5.371765e-01 1.846588e+00 3.040706e+00
##   [71] 2.545882e-01 1.464235e+00 8.016471e-01 1.821176e+00 1.178118e+00
##   [76] 1.852235e+00 1.452235e+00 9.475294e-01 3.465176e+00 9.230588e-01
##   [81] 3.875294e-01 8.957647e-01 1.771294e+00 5.524706e-01 1.060706e+00
##   [86] 5.483059e+00 2.773412e+00 2.912471e+00 2.810118e+00 3.799294e+00
##   [91] 1.140706e+00 3.564706e-01 3.309882e+00 2.156941e+00 2.856000e+00
```

```
##   [96] 8.665882e-01 1.140706e+00 2.689882e+00 4.778118e+00 1.841647e+00
##  [101] 5.341176e-02 2.276471e+00 5.961882e+00 2.964706e-01 6.498824e-01
##  [106] 1.058824e-01 1.794588e+00 9.251765e-01 4.478824e+00 3.066353e+00
##  [111] 1.573412e+00 2.703529e+00 2.877647e-01 2.759294e+00 6.981647e+00
##  [116] 6.914588e+00 5.654118e-01 5.679529e+00 2.488941e+00 2.150118e+00
##  [121] 4.194588e+00 5.717647e-02 2.709412e+00 3.169412e-01 3.755765e+00
##  [126] 5.682353e-01 3.478353e+00 8.117647e-01 2.411059e+00 2.856000e+00
##  [131] 2.778118e+00 1.185882e+00 2.804706e-01 3.258118e+00 2.117647e-02
##  [136] 2.431059e+00 9.804706e-01 7.792941e-01 2.338353e+00 8.251765e-01
##  [141] 2.730118e+00 2.664235e+00 1.111294e+00 8.663529e-01 6.077647e-01
##  [146] 1.248706e+00 7.112941e-01 1.051059e+00 2.034588e+00 5.201882e+00
##  [151] 2.186353e+00 2.635294e+00 1.987529e+00 8.195294e-01 3.815294e+00
##  [156] 2.178353e+00 4.334824e+00 1.562824e+00 1.866353e+00 4.593412e+00
##  [161] 8.505882e-01 9.183529e-01 1.591059e+00 2.251059e+00 1.022824e+00
##  [166] 3.770118e+00 2.115765e+00 3.187765e+00 1.796471e+00 2.074824e+00
##  [171] 7.455765e+00 1.252235e+00 5.271059e+00 1.022824e+00 2.902353e+00
##  [176] 9.223529e-02 7.489412e-01 2.102353e+00 4.169412e-01 6.051765e-01
##  [181] 1.650588e+00 4.992941e-01 3.050118e+00 4.270588e-01 2.381647e+00
##  [186] 4.334824e+00 2.245412e+00 1.977176e+00 7.082353e-01 1.387765e+00
##  [191] 5.701176e-01 5.905412e+00 1.037647e+00 1.795294e-01 9.148235e-01
##  [196] 1.372235e+00 5.898824e-01 1.192941e-01 2.265412e+00 1.592235e+00
##  [201] 1.938824e+00 1.277647e-01 1.178824e+00 8.712941e-01 1.426588e+00
##  [206] 5.111059e+00 1.334588e+00 1.130118e+00 2.024235e+00 1.265882e-01
##  [211] 3.372941e+00 1.683247e+01 1.658118e+00 1.772235e+00 3.642353e-01
##  [216] 1.549882e+00 8.094118e-01 3.309882e+00 1.021882e+00 1.016471e-01
##  [221] 8.505882e-01 4.080706e+00 3.972235e+00 8.098824e-01 1.211765e-01
##  [226] 1.054118e-01 1.811294e+00 2.087059e-01 7.329412e-01 1.821882e+00
##  [231] 7.552941e-01 5.748235e-01 1.556471e+00 3.018824e-01 1.167482e+01
##  [236] 7.564706e-01 4.957176e+00 3.821647e+00 6.312941e-01 7.207059e-01
##  [241] 1.924235e+00 8.395294e-01 1.663529e-01 3.298353e+00 3.348235e-01
##  [246] 4.097647e+00 7.576471e-02 3.376471e-01 5.788235e-01 5.463529e-01
##  [251] 7.934118e-01 1.663529e-01 3.691059e+00 4.411294e+00 1.432471e+00
##  [256] 2.705882e-02 2.268941e+00 1.090588e+00 1.227059e+00 2.314118e+00
##  [261] 2.985882e+00 8.195294e-01 2.654588e+00 1.143529e+00 9.804706e-01
##  [266] 4.105412e+00 8.301176e-01 6.281882e+00 2.061882e+00 4.722353e-01
##  [271] 3.260706e+00 7.724706e-01 1.394588e+00 1.782353e+00 1.228235e-01
##  [276] 1.149882e+00 3.163059e+00 1.398353e+00 5.509412e+00 5.929412e-02
##  [281] 1.211765e-01 7.905882e-01 2.345412e+00 2.394824e+00 3.733412e+00
##  [286] 2.862824e+00 2.071059e+00 1.447059e-01 3.616941e+00 4.750118e+00
##  [291] 6.654118e-01 8.050588e+00 1.949882e+00 8.788235e-01 2.322353e-01
##  [296] 3.653647e+00 3.112941e-01 1.460471e+00 7.470588e-01 1.037647e+00
##  [301] 6.611765e-01 6.635294e-02 5.804706e-01 2.313412e+00 1.888706e+00
##  [306] 1.799294e+00 2.435294e-01 3.705176e+00 4.527765e+00 2.386588e+00
##  [311] 2.338353e+00 1.276941e+00 1.415294e+00 9.748235e-01 2.517647e-02
##  [316] 5.710588e-01 8.701176e-01 9.616471e-01 4.196471e+00 3.885176e+00
##  [321] 3.561882e+00 3.392941e-01 3.135765e+00 2.991059e+00 2.272941e+00
##  [326] 1.277647e-01 4.722353e-01 7.235294e-01 9.063529e-01 6.997647e+00
##  [331] 4.665882e-01 6.670588e-01 3.912941e-01 2.331059e+00 4.235294e-03
##  [336] 3.462353e+00 3.388941e+00 1.972471e+00 6.494118e-01 1.233412e+00
##  [341] 1.992235e+00 1.833412e+00 6.658824e-02 1.200706e+00 1.412471e+00
##  [346] 2.982353e+00 3.241882e+00 3.399294e+00 2.261647e+00 2.357647e-01
##  [351] 9.223529e-02 1.230588e-01 3.493412e+00 4.898824e-01 5.199529e+00
##  [356] 1.549882e+00 1.837176e+00 3.614824e+00 2.213647e+00 1.711059e+00
##  [361] 1.285412e+00 6.275294e-01 2.075294e-01 5.731765e+00 4.919294e+00
```

```
##  [366] 2.979529e+00 1.185882e+00 2.831765e+00 2.604706e-01 1.105412e+00
##  [371] 5.289882e+00 2.142824e+00 1.177176e+00 7.082353e-01 8.251765e-01
##  [376] 1.308706e+00 1.204706e-01 1.334118e-01 2.755765e+00 2.792941e-01
##  [381] 8.110588e-01 6.249412e+00 1.051059e+00 3.771765e-01 8.745412e+00
##  [386] 4.235294e-01 6.501176e-01 7.665882e-01 2.743059e+00 4.172941e+00
##  [391] 3.863529e-01 7.122353e-01 9.487059e-01 1.923529e+00 1.144235e+00
##  [396] 1.611059e+00 2.371765e-01 4.628235e-01 5.367529e+00 3.403059e+00
##  [401] 1.372235e+00 1.770118e+00 8.665882e-01 2.928706e+00 1.722824e+00
##  [406] 5.654118e-01 6.275294e-01 1.136471e-01 2.428235e-01 3.653647e+00
##  [411] 2.287059e-01 1.228941e+00 5.117647e-01 1.101882e+00 3.357647e-01
##  [416] 2.734118e-01 1.748235e-01 2.176000e+00 4.016471e-01 1.048941e+00
##  [421] 3.038824e+00 8.886353e+00 1.114824e+00 6.501176e-01 1.736471e-01
##  [426] 1.214588e+00 3.300471e+00 4.191765e+00 2.545412e+00 8.795294e-01
##  [431] 1.898118e+00 1.149882e+00 8.211765e-01 2.274588e+00 4.957176e+00
##  [436] 1.702353e+00 8.995294e-01 4.705882e-04 1.792941e-01 2.804706e-01
##  [441] 5.218824e-01 4.273412e+00 1.357176e+00 5.902824e+00 1.696471e+00
##  [446] 2.651765e-01 3.929412e-02 2.488941e+00 2.371765e-01 4.521176e+00
##  [451] 1.359529e+00 3.417176e+00 1.628706e+00 4.616471e-01 9.552941e-01
##  [456] 2.993412e+00 1.352941e-01 1.696941e+00 5.465882e-01 2.134588e+00
##  [461] 4.051765e-01 6.917647e-01 2.001882e+00 3.448941e+00 9.381176e-01
##  [466] 1.732235e+00 7.652235e+00 1.764235e+00 2.260706e+00 4.545882e-01
##  [471] 2.339529e+00 2.215294e+00 2.209882e+00 5.935765e+00 1.468941e+00
##  [476] 1.136471e-01 2.755765e+00 5.183059e+00 4.305882e-02 5.672235e+00
##  [481] 1.306353e+00 1.762353e+00 2.924000e+00 5.791765e+00 3.233647e+00
##  [486] 2.928706e+00 1.045176e+00 4.494824e+00 1.087765e+00 5.287059e-01
##  [491] 3.278118e+00 9.983529e-01 4.294588e+00 2.386588e+00 7.724706e-01
##  [496] 3.305882e+00 5.488941e+00 2.658353e+00 2.463529e-01 7.451765e-01
##  [501] 3.204706e-01 7.724706e-01 1.282824e+00 5.277176e+00 3.392941e-01
##  [506] 3.348235e-01 3.863529e-01 1.694118e-02 8.828235e-01 4.696941e+00
##  [511] 1.615765e+00 1.507529e+00 5.816471e-01 1.914118e+00 2.557647e-01
##  [516] 1.105412e+00 3.317647e-01 2.877647e-01 1.494118e-01 3.051294e+00
##  [521] 1.507765e+00 9.878353e+00 5.939529e+00 1.061647e+00 5.712941e-01
##  [526] 4.204706e-01 1.037647e+00 2.922353e-01 1.132235e+00 2.545882e-01
##  [531] 1.936941e+00 1.375765e+00 3.239529e+00 2.206588e+00 1.019294e+00
##  [536] 2.124706e-01 1.176471e-02 6.028235e-01 1.281882e+00 1.925412e+00
##  [541] 1.964706e-01 4.825412e+00 2.223529e+00 1.403059e+00 3.177176e+00
##  [546] 1.157647e-01 3.112941e-01 1.139294e+00 1.694824e+00 1.228941e+00
##  [551] 1.765412e+00 4.866353e+00 1.227059e+00 9.204706e-01 1.297647e+00
##  [556] 2.301176e-01 7.595294e-01 1.736471e-01 2.344235e+00 7.455765e+00
##  [561] 3.399529e+00 2.194588e+00 3.141176e-01 1.136471e-01 6.070588e-02
##  [566] 6.282353e-02 1.709882e+00 7.217647e+00 5.145882e-01 8.470588e-02
##  [571] 5.411765e-03 2.629412e+00 3.085882e+00 8.195294e-01 2.206588e+00
##  [576] 1.022824e+00 5.944235e+00 1.344706e+00 3.014118e+00 1.627059e+00
##  [581] 1.691059e+00 6.098824e-01 1.324235e+00 4.807059e-01 6.268941e+00
##  [586] 5.102824e+00 2.322353e-01 8.324706e-01 5.554588e+00 4.957647e-01
##  [591] 7.592941e-01 5.463529e-01 7.514824e+00 3.016471e-01 6.858353e+00
##  [596] 4.339529e+00 5.425412e+00 6.176471e+00 3.267529e+00 6.706588e+00
##  [601] 2.008941e+00 7.077647e-01 1.952471e+00 3.348235e-01 1.116941e+00
##  [606] 4.336941e+00 9.882353e-01 5.925412e+00 7.028235e-01 1.438824e+00
##  [611] 1.146353e+00 1.062353e+00 6.098824e-01 5.248706e+00 1.813412e+00
##  [616] 4.524706e-01 2.971059e+00 2.149412e+00 8.383529e-01 3.861176e+00
##  [621] 2.971059e+00 8.251765e-01 1.664706e+00 1.402353e+00 4.110588e-01
##  [626] 3.985412e+00 2.964706e-01 2.583529e-01 8.383529e-01 2.759294e+00
##  [631] 3.278118e+00 5.701176e-01 3.905882e-01 3.939529e+00 2.646353e+00
```

```
##  [636] 1.524706e-01 5.663529e-01 9.889412e-01 3.995294e-01 1.738118e+00
##  [641] 9.616471e-01 2.406353e+00 8.082353e-01 3.335294e+00 8.875294e-01
##  [646] 2.162824e+00 9.063529e-01 1.482353e+00 1.339294e+00 4.414824e+00
##  [651] 1.004706e-01 9.289412e-01 7.395294e-01 1.338118e+00 3.109412e+00
##  [656] 3.068235e+00 6.687059e-01 7.263529e-01 1.446353e+00 1.647059e-03
##  [661] 3.120706e+00 2.674824e+00 1.513412e+00 7.176471e-01 4.521176e+00
##  [666] 4.047059e-02 1.022824e+00 2.164706e+00 7.369412e-01 4.729412e-01
##  [671] 3.485176e+00 3.426353e+00 2.067529e+00 7.694118e-02 3.177176e+00
##  [676] 1.821882e+00 3.875294e-01 2.386588e+00 3.223059e+00 2.805176e+00
##  [681] 1.835765e+00 2.241647e+00 1.086588e+00 2.413647e+00 1.134118e-01
##  [686] 2.541176e-02 2.922353e-01 1.471059e+00 1.779529e+00 8.350588e+00
##  [691] 5.188235e+00 9.529412e-01 4.383529e-01 8.055294e+00 5.909412e+00
##  [696] 1.854588e+00 5.012471e+00 4.277647e-01 3.604706e-01 4.255765e+00
##  [701] 5.462353e+00 1.286588e+00 2.900706e+00 6.388235e-01 6.510588e-01
##  [706] 2.169412e-01 2.886353e+00 1.206353e+00 1.799294e+00 1.835765e+00
##  [711] 6.282353e-02 1.324235e+00 3.164235e+00 3.577176e+00 5.700471e+00
##  [716] 2.538118e+00 5.929412e-02 3.659294e+00 4.992941e-01 7.924706e-01
##  [721] 3.477176e+00 8.724706e-01 7.648941e+00 4.844235e+00 6.042353e-01
##  [726] 1.192941e-01 2.053412e+00 1.922824e+00 1.380471e+00 1.262824e+00
##  [731] 1.790588e+00 6.136941e+00 5.858824e-01 4.091294e+00 1.093647e+00
##  [736] 1.569412e-01 4.668706e+00 7.722353e-01 8.663529e-01 2.950118e+00
##  [741] 2.999294e+00 2.240471e+00 1.004706e-01 1.524706e-01 4.395294e-01
##  [746] 1.964706e-01 7.552941e-01 2.432471e+00 1.212235e+00 5.929412e-02
##  [751] 1.732235e+00 3.208235e+00 1.339294e+00 3.558118e+00 2.314118e+00
##  [756] 3.737176e+00 4.832471e+00 1.647529e+00 8.682353e-01 8.995294e-01
##  [761] 9.663529e-01 1.604235e+00 9.889412e-01 2.923059e+00 2.127765e+00
##  [766] 6.254118e-01 1.098824e-01 3.294588e+00 2.040000e-01 6.416471e-01
##  [771] 6.875294e-01 1.393412e+00 3.524235e+00 9.917647e-01 3.392941e-01
##  [776] 3.394824e+00 4.199294e+00 1.073412e+00 1.929412e-02 2.727529e+00
##  [781] 2.837647e+00 5.312941e-01 1.782353e+00 9.529412e-01 1.846588e+00
##  [786] 2.072235e+00 1.788706e+00 1.394588e+00 4.098353e+00 2.710588e-01
##  [791] 1.248706e+00 1.620706e+00 5.595294e-01 1.311765e+00 4.855294e+00
##  [796] 2.305882e-02 1.022824e+00 7.536471e-01 3.572471e+00 2.152941e-01
##  [801] 8.395294e-01 2.460706e+00 3.803059e+00 2.922353e-01 1.950588e+00
##  [806] 5.037647e+00 5.929412e-02 5.954588e+00 3.016471e-01 1.976471e-01
##  [811] 3.394588e+00 3.703059e+00 9.616471e-01 1.610118e+00 6.171765e-01
##  [816] 2.675765e+00 1.861647e+00 1.600706e+00 2.460706e+00 1.659294e+00
##  [821] 4.505882e-01 4.456471e+00 6.028235e-01 1.092471e+00 1.924706e-01
##  [826] 2.276471e+00 2.808941e+00 5.287529e+00 4.104235e+00 2.522353e-01
##  [831] 1.426588e+00 4.912941e-01 3.320471e+00 2.411765e+00 4.823529e-01
##  [836] 6.818824e-01 7.687059e-01 3.137647e+00 9.475294e-01 3.981176e-01
##  [841] 2.656941e+00 7.231765e+00 3.769412e-01 1.714588e+00 2.889412e-01
##  [846] 1.235294e+00 1.487765e+00 3.702353e+00 1.976941e+00 7.094118e-01
##  [851] 6.254118e-01 1.517647e+00 1.792941e-01 5.971765e-01 1.621647e+00
##  [856] 1.190118e+00 3.858824e-01 4.334118e-01 1.228941e+00 2.274588e+00
##  [861] 2.668235e+00 1.941647e+00 3.078353e+00 5.682353e-01 2.981176e-01
##  [866] 1.132235e+00 4.871294e+00 1.374824e+00 1.188706e+00 2.099529e+00
##  [871] 6.494118e-01 1.022824e+00 2.429882e+00 1.214118e+00 1.647765e+00
##  [876] 3.176471e-02 1.607529e+00 3.818118e+00 2.651765e-01 3.390118e+00
##  [881] 1.266588e+00 4.798118e+00 3.228706e+00 6.352941e-03 5.413412e+00
##  [886] 1.274824e+00 1.090588e+00 2.452471e+00 2.851765e-01 1.076471e+00
##  [891] 3.771294e+00 4.545882e-01 3.295765e+00 9.035294e-01 2.993647e+00
##  [896] 1.729412e-01 6.077647e-01 1.281882e+00 1.956471e+00 2.769882e+00
##  [901] 1.423059e+00 1.065176e+00 3.459529e+00 1.393412e+00 5.936471e-01
```

```
##  [906] 5.258824e-01 3.775765e+00 1.003059e+00 1.423059e+00 6.416471e-01
##  [911] 7.498824e-01 2.573647e+00 4.160471e+00 5.992941e-01 2.327059e+00
##  [916] 4.101176e-01 8.098824e-01 2.827059e+00 8.663529e-01 1.121647e+00
##  [921] 2.861647e+00 5.709882e+00 7.828235e-01 9.365176e+00 1.265412e+00
##  [926] 2.357647e-01 1.655294e+00 1.173412e+00 3.785882e+00 1.775765e+00
##  [931] 8.682353e-01 2.231294e+00 3.969412e-01 2.228235e-01 1.077176e+00
##  [936] 5.568706e+00 4.493647e+00 3.108706e+00 4.433412e+00 4.665882e-01
##  [941] 9.854118e-01 2.094118e-01 2.808941e+00 1.206353e+00 6.325412e+00
##  [946] 4.876941e+00 4.687059e-01 9.936471e-01 3.878118e+00 2.198353e+00
##  [951] 1.065176e+00 1.019294e+00 3.319529e+00 2.124706e-01 3.228235e-01
##  [956] 1.051059e+00 1.192235e+00 2.947765e+00 8.640471e+00 2.376471e-01
##  [961] 1.178824e+00 3.952941e-02 1.291765e+00 7.623529e-01 3.771765e-01
##  [966] 2.863059e+00 1.040706e+00 6.152941e-01 3.232471e+00 2.213647e+00
##  [971] 9.798353e+00 5.369412e-01 3.208235e+00 6.228235e-01 2.284235e+00
##  [976] 1.278824e+00 3.929412e-02 2.743059e+00 6.231059e+00 4.687765e+00
##  [981] 1.016941e+00 2.654118e-01 5.056471e+00 6.630588e-01 2.266353e+00
##  [986] 3.941176e-01 3.394588e+00 4.080000e+00 7.741176e-01 5.729412e-01
##  [991] 5.210118e+00 1.525412e+00 4.729412e-01 1.924706e-01 1.080471e+00
##  [996] 8.148235e-01 8.847059e-01 1.705176e+00 1.769882e+00 3.705882e-01
```

e.

Write a function which performs the Kruskal-Wallis test using the function in point c. for m repeated experiments and returns a vector of m test statistics. The input of this function are a matrix `X` with m rows and n columns and a vector `g` which gives the grouping.

```r
fast_kw <- apply(X, 1, function(row){kw_fast(row, grp)})
fast_kw
```

```
##    [1] 3.846353e+00 5.463529e-01 3.335294e+00 1.878353e+00 1.508706e+00
##    [6] 2.677176e+00 7.181176e-01 5.186588e+00 2.265882e-01 2.431059e+00
##   [11] 2.058353e+00 5.671294e+00 2.172941e+00 2.488235e+00 3.172235e+00
##   [16] 2.443059e+00 6.416471e-01 1.243059e+00 1.274824e+00 1.821882e+00
##   [21] 2.308235e+00 1.478118e+00 7.771765e-01 4.748235e-01 1.718353e+00
##   [26] 4.371765e-01 1.183482e+01 3.204706e-01 2.191765e+00 4.784706e+00
##   [31] 2.124706e-01 1.711059e+00 2.386588e+00 3.329412e-01 4.623059e+00
##   [36] 9.428235e-01 5.238353e+00 4.259529e+00 8.004706e-01 1.054118e-01
##   [41] 6.818824e-01 2.971765e-01 2.913647e+00 1.607529e+00 1.923529e+00
##   [46] 1.294824e+00 3.664706e+00 8.195294e-01 6.252941e+00 1.003059e+00
##   [51] 5.761882e+00 1.265412e+00 1.729412e-01 1.725882e+00 2.921882e+00
##   [56] 2.894118e-02 7.630118e+00 2.033647e+00 5.465882e-01 5.717647e-02
##   [61] 6.543529e+00 1.167529e+00 6.416471e-01 2.325176e+00 8.658824e-02
##   [66] 4.298824e-01 1.719529e+00 5.371765e-01 1.846588e+00 3.040706e+00
##   [71] 2.545882e-01 1.464235e+00 8.016471e-01 1.821176e+00 1.178118e+00
##   [76] 1.852235e+00 1.452235e+00 9.475294e-01 3.465176e+00 9.230588e-01
##   [81] 3.875294e-01 8.957647e-01 1.771294e+00 5.524706e-01 1.060706e+00
##   [86] 5.483059e+00 2.773412e+00 2.912471e+00 2.810118e+00 3.799294e+00
##   [91] 1.140706e+00 3.564706e-01 3.309882e+00 2.156941e+00 2.856000e+00
##   [96] 8.665882e-01 1.140706e+00 2.689882e+00 4.778118e+00 1.841647e+00
##  [101] 5.341176e-02 2.276471e+00 5.961882e+00 2.964706e-01 6.498824e-01
##  [106] 1.058824e-01 1.794588e+00 9.251765e-01 4.478824e+00 3.066353e+00
##  [111] 1.573412e+00 2.703529e+00 2.877647e-01 2.759294e+00 6.981647e+00
##  [116] 6.914588e+00 5.654118e-01 5.679529e+00 2.488941e+00 2.150118e+00
##  [121] 4.194588e+00 5.717647e-02 2.709412e+00 3.169412e-01 3.755765e+00
##  [126] 5.682353e-01 3.478353e+00 8.117647e-01 2.411059e+00 2.856000e+00
```

```
##  [131] 2.778118e+00 1.185882e+00 2.804706e-01 3.258118e+00 2.117647e-02
##  [136] 2.431059e+00 9.804706e-01 7.792941e-01 2.338353e+00 8.251765e-01
##  [141] 2.730118e+00 2.664235e+00 1.111294e+00 8.663529e-01 6.077647e-01
##  [146] 1.248706e+00 7.112941e-01 1.051059e+00 2.034588e+00 5.201882e+00
##  [151] 2.186353e+00 2.635294e+00 1.987529e+00 8.195294e-01 3.815294e+00
##  [156] 2.178353e+00 4.334824e+00 1.562824e+00 1.866353e+00 4.593412e+00
##  [161] 8.505882e-01 9.183529e-01 1.591059e+00 2.251059e+00 1.022824e+00
##  [166] 3.770118e+00 2.115765e+00 3.187765e+00 1.796471e+00 2.074824e+00
##  [171] 7.455765e+00 1.252235e+00 5.271059e+00 1.022824e+00 2.902353e+00
##  [176] 9.223529e-02 7.489412e-01 2.102353e+00 4.169412e-01 6.051765e-01
##  [181] 1.650588e+00 4.992941e-01 3.050118e+00 4.270588e-01 2.381647e+00
##  [186] 4.334824e+00 2.245412e+00 1.977176e+00 7.082353e-01 1.387765e+00
##  [191] 5.701176e-01 5.905412e+00 1.037647e+00 1.795294e-01 9.148235e-01
##  [196] 1.372235e+00 5.898824e-01 1.192941e-01 2.265412e+00 1.592235e+00
##  [201] 1.938824e+00 1.277647e-01 1.178824e+00 8.712941e-01 1.426588e+00
##  [206] 5.111059e+00 1.334588e+00 1.130118e+00 2.024235e+00 1.265882e-01
##  [211] 3.372941e+00 1.683247e+01 1.658118e+00 1.772235e+00 3.642353e-01
##  [216] 1.549882e+00 8.094118e-01 3.309882e+00 1.021882e+00 1.016471e-01
##  [221] 8.505882e-01 4.080706e+00 3.972235e+00 8.098824e-01 1.211765e-01
##  [226] 1.054118e-01 1.811294e+00 2.087059e-01 7.329412e-01 1.821882e+00
##  [231] 7.552941e-01 5.748235e-01 1.556471e+00 3.018824e-01 1.167482e+01
##  [236] 7.564706e-01 4.957176e+00 3.821647e+00 6.312941e-01 7.207059e-01
##  [241] 1.924235e+00 8.395294e-01 1.663529e-01 3.298353e+00 3.348235e-01
##  [246] 4.097647e+00 7.576471e-02 3.376471e-01 5.788235e-01 5.463529e-01
##  [251] 7.934118e-01 1.663529e-01 3.691059e+00 4.411294e+00 1.432471e+00
##  [256] 2.705882e-02 2.268941e+00 1.090588e+00 1.227059e+00 2.314118e+00
##  [261] 2.985882e+00 8.195294e-01 2.654588e+00 1.143529e+00 9.804706e-01
##  [266] 4.105412e+00 8.301176e-01 6.281882e+00 2.061882e+00 4.722353e-01
##  [271] 3.260706e+00 7.724706e-01 1.394588e+00 1.782353e+00 1.228235e-01
##  [276] 1.149882e+00 3.163059e+00 1.398353e+00 5.509412e+00 5.929412e-02
##  [281] 1.211765e-01 7.905882e-01 2.345412e+00 2.394824e+00 3.733412e+00
##  [286] 2.862824e+00 2.071059e+00 1.447059e-01 3.616941e+00 4.750118e+00
##  [291] 6.654118e-01 8.050588e+00 1.949882e+00 8.788235e-01 2.322353e-01
##  [296] 3.653647e+00 3.112941e-01 1.460471e+00 7.470588e-01 1.037647e+00
##  [301] 6.611765e-01 6.635294e-02 5.804706e-01 2.313412e+00 1.888706e+00
##  [306] 1.799294e+00 2.435294e-01 3.705176e+00 4.527765e+00 2.386588e+00
##  [311] 2.338353e+00 1.276941e+00 1.415294e+00 9.748235e-01 2.517647e-02
##  [316] 5.710588e-01 8.701176e-01 9.616471e-01 4.196471e+00 3.885176e+00
##  [321] 3.561882e+00 3.392941e-01 3.135765e+00 2.991059e+00 2.272941e+00
##  [326] 1.277647e-01 4.722353e-01 7.235294e-01 9.063529e-01 6.997647e+00
##  [331] 4.665882e-01 6.670588e-01 3.912941e-01 2.331059e+00 4.235294e-03
##  [336] 3.462353e+00 3.388941e+00 1.972471e+00 6.494118e-01 1.233412e+00
##  [341] 1.992235e+00 1.833412e+00 6.658824e-02 1.200706e+00 1.412471e+00
##  [346] 2.982353e+00 3.241882e+00 3.399294e+00 2.261647e+00 2.357647e-01
##  [351] 9.223529e-02 1.230588e-01 3.493412e+00 4.898824e-01 5.199529e+00
##  [356] 1.549882e+00 1.837176e+00 3.614824e+00 2.213647e+00 1.711059e+00
##  [361] 1.285412e+00 6.275294e-01 2.075294e-01 5.731765e+00 4.919294e+00
##  [366] 2.979529e+00 1.185882e+00 2.831765e+00 2.604706e-01 1.105412e+00
##  [371] 5.289882e+00 2.142824e+00 1.177176e+00 7.082353e-01 8.251765e-01
##  [376] 1.308706e+00 1.204706e-01 1.334118e-01 2.755765e+00 2.792941e-01
##  [381] 8.110588e-01 6.249412e+00 1.051059e+00 3.771765e-01 8.745412e+00
##  [386] 4.235294e-01 6.501176e-01 7.665882e-01 2.743059e+00 4.172941e+00
##  [391] 3.863529e-01 7.122353e-01 9.487059e-01 1.923529e+00 1.144235e+00
##  [396] 1.611059e+00 2.371765e-01 4.628235e-01 5.367529e+00 3.403059e+00
```

```
##  [401] 1.372235e+00 1.770118e+00 8.665882e-01 2.928706e+00 1.722824e+00
##  [406] 5.654118e-01 6.275294e-01 1.136471e-01 2.428235e-01 3.653647e+00
##  [411] 2.287059e-01 1.228941e+00 5.117647e-01 1.101882e+00 3.357647e-01
##  [416] 2.734118e-01 1.748235e-01 2.176000e+00 4.016471e-01 1.048941e+00
##  [421] 3.038824e+00 8.886353e+00 1.114824e+00 6.501176e-01 1.736471e-01
##  [426] 1.214588e+00 3.300471e+00 4.191765e+00 2.545412e+00 8.795294e-01
##  [431] 1.898118e+00 1.149882e+00 8.211765e-01 2.274588e+00 4.957176e+00
##  [436] 1.702353e+00 8.995294e-01 4.705882e-04 1.792941e-01 2.804706e-01
##  [441] 5.218824e-01 4.273412e+00 1.357176e+00 5.902824e+00 1.696471e+00
##  [446] 2.651765e-01 3.929412e-02 2.488941e+00 2.371765e-01 4.521176e+00
##  [451] 1.359529e+00 3.417176e+00 1.628706e+00 4.616471e-01 9.552941e-01
##  [456] 2.993412e+00 1.352941e-01 1.696941e+00 5.465882e-01 2.134588e+00
##  [461] 4.051765e-01 6.917647e-01 2.001882e+00 3.448941e+00 9.381176e-01
##  [466] 1.732235e+00 7.652235e+00 1.764235e+00 2.260706e+00 4.545882e-01
##  [471] 2.339529e+00 2.215294e+00 2.209882e+00 5.935765e+00 1.468941e+00
##  [476] 1.136471e-01 2.755765e+00 5.183059e+00 4.305882e-02 5.672235e+00
##  [481] 1.306353e+00 1.762353e+00 2.924000e+00 5.791765e+00 3.233647e+00
##  [486] 2.928706e+00 1.045176e+00 4.494824e+00 1.087765e+00 5.287059e-01
##  [491] 3.278118e+00 9.983529e-01 4.294588e+00 2.386588e+00 7.724706e-01
##  [496] 3.305882e+00 5.488941e+00 2.658353e+00 2.463529e-01 7.451765e-01
##  [501] 3.204706e-01 7.724706e-01 1.282824e+00 5.277176e+00 3.392941e-01
##  [506] 3.348235e-01 3.863529e-01 1.694118e-02 8.828235e-01 4.696941e+00
##  [511] 1.615765e+00 1.507529e+00 5.816471e-01 1.914118e+00 2.557647e-01
##  [516] 1.105412e+00 3.317647e-01 2.877647e-01 1.494118e-01 3.051294e+00
##  [521] 1.507765e+00 9.878353e+00 5.939529e+00 1.061647e+00 5.712941e-01
##  [526] 4.204706e-01 1.037647e+00 2.922353e-01 1.132235e+00 2.545882e-01
##  [531] 1.936941e+00 1.375765e+00 3.239529e+00 2.206588e+00 1.019294e+00
##  [536] 2.124706e-01 1.176471e-02 6.028235e-01 1.281882e+00 1.925412e+00
##  [541] 1.964706e-01 4.825412e+00 2.223529e+00 1.403059e+00 3.177176e+00
##  [546] 1.157647e-01 3.112941e-01 1.139294e+00 1.694824e+00 1.228941e+00
##  [551] 1.765412e+00 4.866353e+00 1.227059e+00 9.204706e-01 1.297647e+00
##  [556] 2.301176e-01 7.595294e-01 1.736471e-01 2.344235e+00 7.455765e+00
##  [561] 3.399529e+00 2.194588e+00 3.141176e-01 1.136471e-01 6.070588e-02
##  [566] 6.282353e-02 1.709882e+00 7.217647e+00 5.145882e-01 8.470588e-02
##  [571] 5.411765e-03 2.629412e+00 3.085882e+00 8.195294e-01 2.206588e+00
##  [576] 1.022824e+00 5.944235e+00 1.344706e+00 3.014118e+00 1.627059e+00
##  [581] 1.691059e+00 6.098824e-01 1.324235e+00 4.807059e-01 6.268941e+00
##  [586] 5.102824e+00 2.322353e-01 8.324706e-01 5.554588e+00 4.957647e-01
##  [591] 7.592941e-01 5.463529e-01 7.514824e+00 3.016471e-01 6.858353e+00
##  [596] 4.339529e+00 5.425412e+00 6.176471e+00 3.267529e+00 6.706588e+00
##  [601] 2.008941e+00 7.077647e-01 1.952471e+00 3.348235e-01 1.116941e+00
##  [606] 4.336941e+00 9.882353e-01 5.925412e+00 7.028235e-01 1.438824e+00
##  [611] 1.146353e+00 1.062353e+00 6.098824e-01 5.248706e+00 1.813412e+00
##  [616] 4.524706e-01 2.971059e+00 2.149412e+00 8.383529e-01 3.861176e+00
##  [621] 2.971059e+00 8.251765e-01 1.664706e+00 1.402353e+00 4.110588e-01
##  [626] 3.985412e+00 2.964706e-01 2.583529e-01 8.383529e-01 2.759294e+00
##  [631] 3.278118e+00 5.701176e-01 3.905882e-01 3.939529e+00 2.646353e+00
##  [636] 1.524706e-01 5.663529e-01 9.889412e-01 3.995294e-01 1.738118e+00
##  [641] 9.616471e-01 2.406353e+00 8.082353e-01 3.335294e+00 8.875294e-01
##  [646] 2.162824e+00 9.063529e-01 1.482353e+00 1.339294e+00 4.414824e+00
##  [651] 1.004706e-01 9.289412e-01 7.395294e-01 1.338118e+00 3.109412e+00
##  [656] 3.068235e+00 6.687059e-01 7.263529e-01 1.446353e+00 1.647059e-03
##  [661] 3.120706e+00 2.674824e+00 1.513412e+00 7.176471e-01 4.521176e+00
##  [666] 4.047059e-02 1.022824e+00 2.164706e+00 7.369412e-01 4.729412e-01
```

```
##  [671] 3.485176e+00 3.426353e+00 2.067529e+00 7.694118e-02 3.177176e+00
##  [676] 1.821882e+00 3.875294e-01 2.386588e+00 3.223059e+00 2.805176e+00
##  [681] 1.835765e+00 2.241647e+00 1.086588e+00 2.413647e+00 1.134118e-01
##  [686] 2.541176e-02 2.922353e-01 1.471059e+00 1.779529e+00 8.350588e+00
##  [691] 5.188235e+00 9.529412e-01 4.383529e-01 8.055294e+00 5.909412e+00
##  [696] 1.854588e+00 5.012471e+00 4.277647e-01 3.604706e-01 4.255765e+00
##  [701] 5.462353e+00 1.286588e+00 2.900706e+00 6.388235e-01 6.510588e-01
##  [706] 2.169412e-01 2.886353e+00 1.206353e+00 1.799294e+00 1.835765e+00
##  [711] 6.282353e-02 1.324235e+00 3.164235e+00 3.577176e+00 5.700471e+00
##  [716] 2.538118e+00 5.929412e-02 3.659294e+00 4.992941e-01 7.924706e-01
##  [721] 3.477176e+00 8.724706e-01 7.648941e+00 4.844235e+00 6.042353e-01
##  [726] 1.192941e-01 2.053412e+00 1.922824e+00 1.380471e+00 1.262824e+00
##  [731] 1.790588e+00 6.136941e+00 5.858824e-01 4.091294e+00 1.093647e+00
##  [736] 1.569412e-01 4.668706e+00 7.722353e-01 8.663529e-01 2.950118e+00
##  [741] 2.999294e+00 2.240471e+00 1.004706e-01 1.524706e-01 4.395294e-01
##  [746] 1.964706e-01 7.552941e-01 2.432471e+00 1.212235e+00 5.929412e-02
##  [751] 1.732235e+00 3.208235e+00 1.339294e+00 3.558118e+00 2.314118e+00
##  [756] 3.737176e+00 4.832471e+00 1.647529e+00 8.682353e-01 8.995294e-01
##  [761] 9.663529e-01 1.604235e+00 9.889412e-01 2.923059e+00 2.127765e+00
##  [766] 6.254118e-01 1.098824e-01 3.294588e+00 2.040000e-01 6.416471e-01
##  [771] 6.875294e-01 1.393412e+00 3.524235e+00 9.917647e-01 3.392941e-01
##  [776] 3.394824e+00 4.199294e+00 1.073412e+00 1.929412e-02 2.727529e+00
##  [781] 2.837647e+00 5.312941e-01 1.782353e+00 9.529412e-01 1.846588e+00
##  [786] 2.072235e+00 1.788706e+00 1.394588e+00 4.098353e+00 2.710588e-01
##  [791] 1.248706e+00 1.620706e+00 5.595294e-01 1.311765e+00 4.855294e+00
##  [796] 2.305882e-02 1.022824e+00 7.536471e-01 3.572471e+00 2.152941e-01
##  [801] 8.395294e-01 2.460706e+00 3.803059e+00 2.922353e-01 1.950588e+00
##  [806] 5.037647e+00 5.929412e-02 5.954588e+00 3.016471e-01 1.976471e-01
##  [811] 3.394588e+00 3.703059e+00 9.616471e-01 1.610118e+00 6.171765e-01
##  [816] 2.675765e+00 1.861647e+00 1.600706e+00 2.460706e+00 1.659294e+00
##  [821] 4.505882e-01 4.456471e+00 6.028235e-01 1.092471e+00 1.924706e-01
##  [826] 2.276471e+00 2.808941e+00 5.287529e+00 4.104235e+00 2.522353e-01
##  [831] 1.426588e+00 4.912941e-01 3.320471e+00 2.411765e+00 4.823529e-01
##  [836] 6.818824e-01 7.687059e-01 3.137647e+00 9.475294e-01 3.981176e-01
##  [841] 2.656941e+00 7.231765e+00 3.769412e-01 1.714588e+00 2.889412e-01
##  [846] 1.235294e+00 1.487765e+00 3.702353e+00 1.976941e+00 7.094118e-01
##  [851] 6.254118e-01 1.517647e+00 1.792941e-01 5.971765e-01 1.621647e+00
##  [856] 1.190118e+00 3.858824e-01 4.334118e-01 1.228941e+00 2.274588e+00
##  [861] 2.668235e+00 1.941647e+00 3.078353e+00 5.682353e-01 2.981176e-01
##  [866] 1.132235e+00 4.871294e+00 1.374824e+00 1.188706e+00 2.099529e+00
##  [871] 6.494118e-01 1.022824e+00 2.429882e+00 1.214118e+00 1.647765e+00
##  [876] 3.176471e-02 1.607529e+00 3.818118e+00 2.651765e-01 3.390118e+00
##  [881] 1.266588e+00 4.798118e+00 3.228706e+00 6.352941e-03 5.413412e+00
##  [886] 1.274824e+00 1.090588e+00 2.452471e+00 2.851765e-01 1.076471e+00
##  [891] 3.771294e+00 4.545882e-01 3.295765e+00 9.035294e-01 2.993647e+00
##  [896] 1.729412e-01 6.077647e-01 1.281882e+00 1.956471e+00 2.769882e+00
##  [901] 1.423059e+00 1.065176e+00 3.459529e+00 1.393412e+00 5.936471e-01
##  [906] 5.258824e-01 3.775765e+00 1.003059e+00 1.423059e+00 6.416471e-01
##  [911] 7.498824e-01 2.573647e+00 4.160471e+00 5.992941e-01 2.327059e+00
##  [916] 4.101176e-01 8.098824e-01 2.827059e+00 8.663529e-01 1.121647e+00
##  [921] 2.861647e+00 5.709882e+00 7.828235e-01 9.365176e+00 1.265412e+00
##  [926] 2.357647e-01 1.655294e+00 1.173412e+00 3.785882e+00 1.775765e+00
##  [931] 8.682353e-01 2.231294e+00 3.969412e-01 2.228235e-01 1.077176e+00
##  [936] 5.568706e+00 4.493647e+00 3.108706e+00 4.433412e+00 4.665882e-01
```

```
##  [941] 9.854118e-01 2.094118e-01 2.808941e+00 1.206353e+00 6.325412e+00
##  [946] 4.876941e+00 4.687059e-01 9.936471e-01 3.878118e+00 2.198353e+00
##  [951] 1.065176e+00 1.019294e+00 3.319529e+00 2.124706e-01 3.228235e-01
##  [956] 1.051059e+00 1.192235e+00 2.947765e+00 8.640471e+00 2.376471e-01
##  [961] 1.178824e+00 3.952941e-02 1.291765e+00 7.623529e-01 3.771765e-01
##  [966] 2.863059e+00 1.040706e+00 6.152941e-01 3.232471e+00 2.213647e+00
##  [971] 9.798353e+00 5.369412e-01 3.208235e+00 6.228235e-01 2.284235e+00
##  [976] 1.278824e+00 3.929412e-02 2.743059e+00 6.231059e+00 4.687765e+00
##  [981] 1.016941e+00 2.654118e-01 5.056471e+00 6.630588e-01 2.266353e+00
##  [986] 3.941176e-01 3.394588e+00 4.080000e+00 7.741176e-01 5.729412e-01
##  [991] 5.210118e+00 1.525412e+00 4.729412e-01 1.924706e-01 1.080471e+00
##  [996] 8.148235e-01 8.847059e-01 1.705176e+00 1.769882e+00 3.705882e-01
```

f.

Compare the performance of the two approaches using a benchmarking package on the data generated above. Comment on the results.

```r
bench_results_small <- bench::mark(
  stats_version = apply(X, 1, function(row) {
    stats:::kruskal.test.default(row, grp)$statistic
  }),
  fast_version = apply(X, 1, function(row) {
    kw_fast(row, grp)
  }),
  check = TRUE
)
```

```
## Warning: Some expressions had a GC in every iteration; so filtering is
## disabled.
```

```r
print(bench_results_small)
```

```
## # A tibble: 2 x 13
##   expression      min median `itr/sec` mem_alloc `gc/sec` n_itr  n_gc total_time
##   <bch:expr>    <bch> <bch:>     <dbl> <bch:byt>    <dbl> <int> <dbl>   <bch:tm>
## 1 stats_version 766ms  766ms      1.31    20.4MB     3.92     1     3      766ms
## 2 fast_version  235ms  241ms      4.13    21.6MB     9.65     3     7      726ms
## # i 4 more variables: result <list>, memory <list>, time <list>, gc <list>
```

```r
rows <- 100
cols <- 50000
X <- matrix(rnorm(rows * cols), nrow = rows)
grp <- factor(grp <- rep(1:3, c(20000, 20000, 10000)))
bench_results_large <- bench::mark(
  stats_version = apply(X, 1, function(row) {
    stats:::kruskal.test.default(row, grp)$statistic
  }),
  fast_version = apply(X, 1, function(row) {
    kw_fast(row, grp)
  }),
  check = TRUE
)
```

```
## Warning: Some expressions had a GC in every iteration; so filtering is
## disabled.
```

```r
print(bench_results_large)
```

```
## # A tibble: 2 x 13
##   expression      min median `itr/sec` mem_alloc `gc/sec` n_itr  n_gc total_time
##   <bch:expr>   <bch:> <bch:>     <dbl> <bch:byt>    <dbl> <int> <dbl>   <bch:tm>
## 1 stats_versi~ 41.77s 41.77s    0.0239    1.68GB    0.551     1    23     41.77s
## 2 fast_version  1.53s  1.53s    0.655   421.28MB    3.27      1     5      1.53s
## # i 4 more variables: result <list>, memory <list>, time <list>, gc <list>
```

The fast version performs at about 1.5 times the speed of the stats version, but runtime is still limited by the apply() loop as well as the relatively small amount of data. The memory allocation shows that the fast version needed more memory than the base version, which is likely due to the data.table and collapse used, as they are made for big data and need RAM to be set up.

To understand if the difference would increase with a larger matrix, where apply becomes less significant, I tested with a 50 000 columns matrix. The fast version is now about 15 times faster than the basic version and uses 4 times less RAM.

g.

Now consider vectorizing the function in point c. Compare this approach to the other two. Comment on the results.

```r
kw_matrix <- function(X, g){
     if (ncol(X) != length(g) | ncol(X) == 0)
            stop("'X'does not have the correct size for given 'g'")
    g <- factor(g)
    k <- nlevels(g)
    if (k < 2L)
        stop("all observations are in the same group")

    n <- ncol(X)
    if (n < 2L)
        stop("not enough observations")

    r <- matrixStats::rowRanks(X, ties.method = "average")
    g_sums <- t(collapse::fsum(t(r), g))
    g_sizes <- collapse::fcount(g)$N

    H <- (12 / (n * (n + 1))) * rowSums(sweep(g_sums^2, 2, g_sizes, "/")) - 3 * (n + 1)


    obs_var <- matrixStats::rowVars(r)
    var <- n * (n + 1) / 12
    tie_correction <- obs_var / var
    return(H / tie_correction)
}

rows <- 100
cols <- 50000
X <- matrix(rnorm(rows * cols), nrow = rows)
grp <- factor(grp <- rep(1:3, c(20000, 20000, 10000)))
bench_results_matrix <- bench::mark(
  stats_version = apply(X, 1, function(row) {
    stats:::kruskal.test.default(row, grp)$statistic
```

```r
  }),
  fast_version = apply(X, 1, function(row) {
    kw_fast(row, grp)
  }),
  matrix_version = kw_matrix(X, grp),
  check = TRUE
)
```

```
## Warning: Some expressions had a GC in every iteration; so filtering is
## disabled.
```

```r
print(bench_results_matrix)
```

```
## # A tibble: 3 x 13
##   expression          min   median `itr/sec` mem_alloc `gc/sec` n_itr  n_gc
##   <bch:expr>     <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl> <int> <dbl>
## 1 stats_version    41.47s   41.47s    0.0241    1.67GB    0.603     1    25
## 2 fast_version      1.63s    1.63s    0.613   421.29MB    3.07      1     5
## 3 matrix_version 476.38ms 479.81ms    2.08     79.28MB    2.08      2     2
## # i 5 more variables: total_time <bch:tm>, result <list>, memory <list>,
## #   time <list>, gc <list>
```

To make the function work, we had to switch our correction approach and now work with the variance. The complete vectorization was nearly twice as fast as the fast version before, as it operated on all rows at once. The largest improvement can be observed in the memory allocation, where only one fifth of the space is used, as there are less intermediate results produced.